

# Chapter 6 — Difference Array

## 0. What You Will Learn in This Chapter

Difference Array is a powerful pattern for problems involving **many range updates**.

Prefix Sum helps when you need many range queries.

Difference Array helps when you need many range updates.

By the end of this chapter, you should understand:

- What a difference array is
- Why difference array exists
- How it is related to prefix sum
- How to apply range updates in  $O(1)$
- How to reconstruct the final array using prefix sum
- When to use difference array
- When not to use difference array
- How to handle inclusive ranges
- How to handle boundary issues
- How to solve range update problems efficiently
- How difference array appears in interval/event problems
- How to extend the idea to strings and 2D grids

Core LeetCode-style problems covered in this chapter:

- 1.1. Range Addition, premium-style classic
  - 2.2. Corporate Flight Bookings
  - 3.3. Car Pooling
  - 4.4. Shifting Letters II
  - 5.5. Maximum Population Year
  - 6.6. Check if All the Integers in a Range Are Covered
  - 7.7. Minimum Number of K Consecutive Bit Flips
  - 8.8. Meeting Rooms II, sweep-line overlap preview
  - 9.9. My Calendar II, sweep-line preview
  - 10.10. Minimum Moves to Make Array Complementary, harder
  - 11.11. Stamping the Grid, advanced 2D prefix/difference style preview
- 

## 1. What Is a Difference Array?

A Difference Array is an array that stores the **change between consecutive elements**.

Given:

```
nums = {5, 7, 10, 10, 6}
```

The difference array diff can be:

```
diff[0] = nums[0]
diff[1] = nums[1] - nums[0]
diff[2] = nums[2] - nums[1]
diff[3] = nums[3] - nums[2]
diff[4] = nums[4] - nums[3]
```

So:

$\text{diff} = \{5, 2, 3, 0, -4\}$

If you take prefix sum of diff, you get back nums:

5

$5 + 2 = 7$

$5 + 2 + 3 = 10$

$5 + 2 + 3 + 0 = 10$

$5 + 2 + 3 + 0 - 4 = 6$

So the key relationship is:

Original array = prefix sum of difference array

## 2. Why Difference Array Exists

Suppose you have an array of size  $n$ .

You need to apply many range updates like:

Add 5 to every element from index 2 to index 7.

Add 3 to every element from index 0 to index 4.

Subtract 2 from every element from index 5 to index 9.

Naive approach:

For every update  $[l, r, \text{val}]$ , loop from  $l$  to  $r$ :

```
for (int i = l; i <= r; i++) {  
    nums[i] += val;  
}
```

If there are  $m$  updates and each range can be size  $n$ , time can be:

$O(m * n)$

Difference array improves this.

Instead of updating every element inside  $[l, r]$ , we only mark:

```
diff[l] += val;  
diff[r + 1] -= val;
```

Then one final prefix sum reconstructs all updates.

Each update becomes  $O(1)$ .

Total:

$O(m + n)$

## 3. Core Idea

To add  $\text{val}$  to every index from  $l$  to  $r$  inclusive:

```
diff[l] += val;  
diff[r + 1] -= val;
```

Why?

`diff[l] += val` means:

Starting from index `l`, add `val`.

`diff[r + 1] -= val` means:

Starting after index `r`, stop adding `val`.

Then prefix sum spreads the effect across the range.

---

## Example

Suppose:

`n = 5`

initial array = `{0, 0, 0, 0, 0}`

Update:

Add 3 from index 1 to index 3.

Use diff of size `n + 1`:

`diff[1] += 3;`

`diff[4] -= 3;`

Now:

`diff = {0, 3, 0, 0, -3, 0}`

Prefix sum:

| 0 | 0 | 0 |
|---|---|---|
| 1 | 3 | 3 |
| 2 | 3 | 3 |
| 3 | 3 | 3 |
| 4 | 0 | 0 |

Final:

`{0, 3, 3, 3, 0}`

Exactly what we wanted.

---

## 4. Difference Array vs Prefix Sum

These two patterns are closely related.

| Prefix Sum       | Fast range queries | Original array → prefix array      |
|------------------|--------------------|------------------------------------|
| Difference Array | Fast range updates | Updates → diff array → final array |

Prefix Sum answers:

What is the sum from l to r?

Difference Array answers:

After many range updates, what does the final array look like?

## Prefix Sum Example

Static array:

nums = {1, 2, 3, 4}

Many range sum queries.

Use prefix sum.

## Difference Array Example

Initial array:

nums = {0, 0, 0, 0}

Many range add updates.

Use difference array.

# 5. Core Need / Why Use Difference Array Over Other Patterns?

Difference Array is used when:

- updates are over ranges
- many updates exist
- queries are mostly after all updates
- each update affects a continuous interval

## 5.1 Why Not Basic Traversal?

Basic traversal applies every update one element at a time.

If there are many updates, this becomes too slow.

Difference array marks only two points per update.

## 5.2 Why Not Prefix Sum Only?

Prefix sum helps answer range sums quickly, but it does not directly optimize range updates.

Difference array is like the reverse tool:

- Prefix sum speeds up range queries.
- Difference array speeds up range updates.

## 5.3 Why Not Segment Tree?

Segment tree is useful when updates and queries are mixed online.

Example:

update range

query range

update range  
query range

Difference array is best when:

apply all updates first  
then compute final result

If you need dynamic updates and queries interleaved, use:

- Fenwick Tree
- Segment Tree
- Lazy Segment Tree

## 5.4 Why Not Sweep Line?

Difference array and sweep line are cousins.

If the coordinate range is small and integer-indexed, difference array is simple.

If coordinates are huge or sparse, use sweep line with map/events.

Example:

Years from 1950 to 2050: difference array is great.

Coordinates up to  $10^9$ : use map/sweep line or coordinate compression.

# 6. When to Use Difference Array

Use this pattern when the problem says:

## 6.1 Many Range Updates

Problem gives operations like:

[l, r, value]

and asks for final array.

Examples:

- Range Addition
- Corporate Flight Bookings
- Shifting Letters II

## 6.2 Add/Subtract Over Intervals

Problem involves intervals where something starts and stops.

Examples:

- Car Pooling
- Maximum Population Year
- Meeting Rooms II
- My Calendar II

## 6.3 Final Result After All Operations

Problem asks:

After all operations, return final array/string/count.

Difference array is ideal.

## 6.4 Coverage Problems

Problem asks whether all positions in a range are covered.

Example:

- Check if All the Integers in a Range Are Covered
- 

## 6.5 Small Fixed Coordinate Range

If years range from 1950 to 2050, or positions range from 0 to 1000, difference array is easy.

---

# 7. When NOT to Use Difference Array

---

## 7.1 Online Queries Mixed With Updates

If you need:

update  
query  
update  
query

Difference array alone is not enough.

Use:

- Fenwick Tree
  - Segment Tree
  - Lazy Segment Tree
- 

## 7.2 Sparse Huge Coordinates

Example:

Intervals involve positions up to  $10^9$ .

A difference array of size  $10^9$  is impossible.

Use:

- ordered map events
  - sweep line
  - coordinate compression
- 

## 7.3 Non-Contiguous Updates

Difference array works for continuous ranges.

If updates affect arbitrary scattered indices, use direct updates, hash maps, or other structures.

---

## 7.4 Range Min/Max Queries

Difference array is not for querying minimum/maximum dynamically.

Use segment tree or monotonic structures.

---

## 8. Mental Models

### 8.1 Start and Stop Signal

For update  $[l, r, val]$ :

```
diff[l] += val;  
diff[r + 1] -= val;
```

Think:

- At  $l$ , turn effect ON.
- At  $r + 1$ , turn effect OFF.

### 8.2 Water Hose Model

Imagine spraying water from index  $l$  to  $r$ .

At  $l$ , open the hose.

At  $r + 1$ , close the hose.

Prefix sum tells how much water is active at each point.

### 8.3 Event Timeline

Each range update is two events:

```
start event: +val  
end event: -val
```

Processing prefix sum over time gives active value.

This is the same idea as sweep line.

### 8.4 Compression of Repeated Work

Instead of touching every element in a range, difference array only records where the effect changes.

## 9. Pattern Recognition Checklist

Ask:

2. Are there many range updates?
3. Does each update affect all elements from  $l$  to  $r$ ?
4. Do I only need final result after all updates?
5. Can I mark start and end of each effect?
6. Are coordinates small enough for an array?
7. Is the range inclusive?
8. Do I need to stop the effect at  $r + 1$ ?
9. Are updates additive?
10. Is this really a sweep-line event problem?
11. Would direct simulation be  $O(n * m)$ ?

If yes, Difference Array is likely useful.

## 10. C++ Tools Needed

## 10.1 vector<int> diff

```
vector<int> diff(n + 1, 0);
```

Often size  $n + 1$  is used so  $r + 1$  is safe.

## 10.2 long long

Use if values or total updates can be large.

```
vector<long long> diff(n + 1, 0);
```

## 10.3 Prefix Reconstruction

```
int running = 0;
```

```
for (int i = 0; i < n; i++) {  
    running += diff[i];  
    result[i] = running;  
}
```

## 10.4 Boundary Check

If using diff size  $n$  instead of  $n + 1$ , then check:

```
if (r + 1 < n) {  
    diff[r + 1] -= val;  
}
```

But using size  $n + 1$  often avoids this.

# 11. Core Templates

## Template 1 — Basic Difference Array for Initially Zero Array

```
vector<int> diff(n + 1, 0);
```

```
for (auto update : updates) {  
    int l = update[0];  
    int r = update[1];  
    int val = update[2];  
  
    diff[l] += val;  
    diff[r + 1] -= val;  
}
```

```
vector<int> result(n);
```

```
int running = 0;

for (int i = 0; i < n; i++) {
    running += diff[i];
    result[i] = running;
}
```

## Template 2 — Difference Array With Existing Initial Array

If initial array is not all zero, you can either:

12. Build diff from original array.
13. Apply updates.
14. Reconstruct.

```
vector<int> diff(n + 1, 0);

diff[0] = nums[0];

for (int i = 1; i < n; i++) {
    diff[i] = nums[i] - nums[i - 1];
}

for (auto update : updates) {
    int l = update[0];
    int r = update[1];
    int val = update[2];

    diff[l] += val;
    diff[r + 1] -= val;
}

vector<int> result(n);
int running = 0;

for (int i = 0; i < n; i++) {
    running += diff[i];
    result[i] = running;
}
```

## Template 3 — Range Count / Coverage

If every interval adds coverage count:

```
vector<int> diff(maxCoord + 2, 0);

for (auto interval : intervals) {
```

```

int l = interval[0];
int r = interval[1];

diff[l] += 1;
diff[r + 1] -= 1;
}

int active = 0;

for (int i = 0; i <= maxCoord; i++) {
    active += diff[i];

    if (active > 0) {
        // i is covered
    }
}

```

## Template 4 — Timeline Events

For intervals [start, end) where end is exclusive:

```

diff[start] += val;
diff[end] -= val;

```

For intervals [start, end] where end is inclusive:

```

diff[start] += val;
diff[end + 1] -= val;

```

This distinction is very important.

## Template 5 — String Shift Difference

For shifting letters:

```

vector<int> diff(n + 1, 0);

for (auto shift : shifts) {
    int l = shift[0];
    int r = shift[1];
    int dir = shift[2];

    int val = (dir == 1) ? 1 : -1;

    diff[l] += val;
    diff[r + 1] -= val;
}

```

```

int running = 0;

for (int i = 0; i < n; i++) {
    running += diff[i];

    int shiftAmount = ((running % 26) + 26) % 26;

    s[i] = 'a' + (s[i] - 'a' + shiftAmount) % 26;
}

```

## Template 6 — 2D Difference Array Preview

For updating rectangle (r1, c1) to (r2, c2) by val:

```

diff[r1][c1] += val;
diff[r2 + 1][c1] -= val;
diff[r1][c2 + 1] -= val;
diff[r2 + 1][c2 + 1] += val;

```

Then reconstruct using 2D prefix sum.

This is advanced, but useful for grid update problems.

# 12. Major Variations of Difference Array

|                      |                        |                               |
|----------------------|------------------------|-------------------------------|
|                      |                        |                               |
| Basic range addition | add val to [l,r]       | Range Addition                |
| Range count          | count active intervals | Maximum Population Year       |
| Capacity check       | active passengers      | Car Pooling                   |
| String range shift   | shift letters          | Shifting Letters II           |
| Coverage check       | covered integers       | Check if All Integers Covered |
| Sweep-line array     | small timeline         | Meeting rooms / population    |
| 2D difference        | rectangle updates      | stamping / grid updates       |
| Greedy diff tracking | active flips           | K Consecutive Bit Flips       |

# 13. Full Problem Walkthrough 1 — Range Addition

This is a classic premium-style problem and the purest difference-array example.

## 13.1 Problem Summary

You are given an integer length and a list of updates.

Each update is:

```
[startIndex, endIndex, inc]
```

You need to apply each update by adding inc to every element from startIndex to endIndex.

Initial array is all zero.

Return final array.

Example:

```
length = 5
updates = {
  {1, 3, 2},
  {2, 4, 3},
  {0, 2, -2}
}
```

## 13.2 How to Identify This Pattern

Problem gives:

- initial array
- many range updates
- add value to every index in range
- final array after all updates

This is exactly Difference Array.

## 13.3 Brute Force Idea

For every update, loop through the range.

```
for each update:
  for i from l to r:
    nums[i] += val
```

If length = n and there are m updates:

Time:  $O(n * m)$

Too slow.

## 13.4 Optimized Intuition

For update [l, r, val]:

```
diff[l] += val;
diff[r + 1] -= val;
```

Then prefix sum reconstructs final array.

---

## 13.5 C++ Code

```
vector<int> getModifiedArray(int length, vector<vector<int>>& updates) {
    vector<int> diff(length + 1, 0);

    for (auto& update : updates) {
        int l = update[0];
        int r = update[1];
        int val = update[2];

        diff[l] += val;
        diff[r + 1] -= val;
    }

    vector<int> result(length);
    int running = 0;

    for (int i = 0; i < length; i++) {
        running += diff[i];
        result[i] = running;
    }

    return result;
}
```

## 13.6 Dry Run

```
length = 5
updates = {
    {1, 3, 2},
    {2, 4, 3},
    {0, 2, -2}
}
```

Initial:

```
diff = {0,0,0,0,0,0}
```

Update {1,3,2}:

```
diff[1] += 2
diff[4] -= 2
diff = {0,2,0,0,-2,0}
```

Update {2,4,3}:

```
diff[2] += 3
diff[5] -= 3
diff = {0,2,3,0,-2,-3}
```

Update {0,2,-2}:

```
diff[0] += -2
diff[3] -= -2
diff = {-2,2,3,2,-2,-3}
```

Prefix reconstruction:

|   |    |    |
|---|----|----|
|   |    |    |
| 0 | -2 | -2 |
| 1 | 0  | 0  |
| 2 | 3  | 3  |
| 3 | 5  | 5  |
| 4 | 3  | 3  |

Final:

```
{-2, 0, 3, 5, 3}
```

## 13.7 Complexity

Time:

```
O(m + n)
```

where m is number of updates and n is length.

Space:

```
O(n)
```

## 13.8 Edge Cases

- update starts at 0
- update ends at last index
- negative update values
- no updates
- single-element array

## 13.9 Common Mistake

Using diff of size length and doing:

```
diff[r + 1] -= val;
```

This can go out of bounds when  $r == \text{length} - 1$ .

Either use size length + 1, or check boundary.

---

## 13.10 Similar Problems

- Corporate Flight Bookings
  - Shifting Letters II
  - Car Pooling
  - Maximum Population Year
- 

# 14. Full Problem Walkthrough 2 — LeetCode 1109: Corporate Flight Bookings

## 14.1 Problem Summary

There are  $n$  flights labeled from 1 to  $n$ .

Each booking is:

[first, last, seats]

This means seats are booked on every flight from first to last.

Return number of seats booked for each flight.

Example:

```
bookings = {  
    {1, 2, 10},  
    {2, 3, 20},  
    {2, 5, 25}  
}  
n = 5
```

Output:

{10, 55, 45, 25, 25}

## 14.2 How to Identify This Pattern

Problem gives:

- many bookings
- each booking updates a range of flights
- return final seats per flight

This is Difference Array.

---

## 14.3 Indexing Detail

Flights are 1-indexed.

C++ arrays are 0-indexed.

So:

```
first flight -> first - 1  
last flight -> last - 1
```

For booking [first, last, seats]:

```
l = first - 1  
r = last - 1
```

Then:

```
diff[l] += seats;  
diff[r + 1] -= seats;
```

## 14.4 C++ Code

```
class Solution {  
public:  
    vector<int> corpFlightBookings(vector<vector<int>>& bookings, int n) {  
        vector<int> diff(n + 1, 0);  
  
        for (auto& booking : bookings) {  
            int first = booking[0];  
            int last = booking[1];  
            int seats = booking[2];  
  
            int l = first - 1;  
            int r = last - 1;  
  
            diff[l] += seats;  
            diff[r + 1] -= seats;  
        }  
  
        vector<int> answer(n);  
        int running = 0;  
  
        for (int i = 0; i < n; i++) {  
            running += diff[i];  
            answer[i] = running;  
        }  
  
        return answer;  
    }  
};
```

## 14.5 Dry Run

```
bookings = {  
    {1, 2, 10},
```

```
{2, 3, 20},  
{2, 5, 25}  
}  
n = 5
```

Initial:

```
diff = {0,0,0,0,0,0}
```

Booking {1,2,10}:

```
l = 0, r = 1  
diff[0] += 10  
diff[2] -= 10  
diff = {10,0,-10,0,0,0}
```

Booking {2,3,20}:

```
l = 1, r = 2  
diff[1] += 20  
diff[3] -= 20  
diff = {10,20,-10,-20,0,0}
```

Booking {2,5,25}:

```
l = 1, r = 4  
diff[1] += 25  
diff[5] -= 25  
diff = {10,45,-10,-20,0,-25}
```

Prefix:

|   |    |    |
|---|----|----|
|   |    |    |
| 0 | 10 | 10 |
| 1 | 55 | 55 |
| 2 | 45 | 45 |
| 3 | 25 | 25 |
| 4 | 25 | 25 |

Output:

```
{10, 55, 45, 25, 25}
```

## 14.6 Complexity

Time:

```
O(bookings.size() + n)
```

Space:

$O(n)$

## 14.7 Edge Cases

- booking starts at flight 1
- booking ends at flight  $n$
- one flight only
- overlapping bookings
- many bookings on same range

## 14.8 Common Mistakes

Mistake 1: Forgetting 1-indexed to 0-indexed conversion

Wrong:

```
diff[first] += seats;
```

Correct:

```
diff[first - 1] += seats;
```

Mistake 2: Not using  $n + 1$

Needed for  $r + 1$ .

## 14.9 Similar Problems

- Range Addition
- Car Pooling
- Shifting Letters II

# 15. Full Problem Walkthrough 3 — LeetCode 1094: Car Pooling

## 15.1 Problem Summary

You are given trips:

```
[numPassengers, from, to]
```

This means passengers get in at `from` and get out at `to`.

Return true if the car capacity is never exceeded.

Important:

The trip interval is:

```
[from, to)
```

Passengers are inside the car from `from` up to but not including `to`.

Example:

```
trips = {  
  {2, 1, 5},  
  {3, 3, 7}}
```

```
}  
capacity = 4
```

Output:

```
false
```

Because between location 3 and 5, passengers = 5.

## 15.2 How to Identify This Pattern

Problem gives:

- intervals along route
- passengers enter and leave
- check active count over coordinate line

This is difference array / sweep line.

## 15.3 Key Detail: End Is Exclusive

For trip:

```
[numPassengers, from, to]
```

Use:

```
diff[from] += passengers;  
diff[to] -= passengers;
```

Not to + 1, because passengers leave at to.

## 15.4 C++ Code

```
class Solution {  
public:  
    bool carPooling(vector<vector<int>>& trips, int capacity) {  
        vector<int> diff(1001, 0);  
  
        for (auto& trip : trips) {  
            int passengers = trip[0];  
            int from = trip[1];  
            int to = trip[2];  
  
            diff[from] += passengers;  
            diff[to] -= passengers;  
        }  
  
        int currentPassengers = 0;  
  
        for (int i = 0; i < diff.size(); i++) {  
            currentPassengers += diff[i];  
        }  
    }  
};
```

```

        if (currentPassengers > capacity) {
            return false;
        }
    }

    return true;
}
};

```

## 15.5 Why Size 1001?

LeetCode constraints usually have locations from 0 to 1000.

So:

```
vector<int> diff(1001, 0);
```

is enough.

If constraints were larger, compute max coordinate or use map.

## 15.6 Dry Run

```

trips = {
    {2, 1, 5},
    {3, 3, 7}
}
capacity = 4

```

Events:

```

diff[1] += 2
diff[5] -= 2

diff[3] += 3
diff[7] -= 3

```

Timeline:

|   |    |   |
|---|----|---|
|   |    |   |
| 1 | +2 | 2 |
| 3 | +3 | 5 |
| 5 | -2 | 3 |
| 7 | -3 | 0 |

At location 3, active passengers = 5 > 4.

Return false.

## 15.7 Complexity

Time:

```
O(trips.size() + 1001)
```

Space:

```
O(1001)
```

More generally:

```
O(m + maxCoordinate)
```

## 15.8 Edge Cases

- passengers leave and others enter at same location
- capacity exactly reached
- no trips
- overlapping trips
- non-overlapping trips

## 15.9 Common Mistakes

Mistake 1: Treating to as inclusive

Wrong:

```
diff[to + 1] -= passengers;
```

Correct:

```
diff[to] -= passengers;
```

Because passengers get out at to.

Mistake 2: Checking only trip endpoints manually

Difference array handles all active counts.

## 15.10 Similar Problems

- Corporate Flight Bookings
- Meeting Rooms II
- Maximum Population Year
- My Calendar II

# 16. Full Problem Walkthrough 4 — LeetCode 2381: Shifting Letters II

## 16.1 Problem Summary

You are given a string *s* and shift operations.

Each shift is:

```
[start, end, direction]
```

If direction == 1, shift letters forward by 1.

If direction == 0, shift letters backward by 1.

Apply all shifts and return final string.

Example:

```
s = "abc"
shifts = {
    {0, 1, 0},
    {1, 2, 1},
    {0, 2, 1}
}
```

## 16.2 How to Identify This Pattern

Problem gives:

- many range operations on string indices
- each operation adds +1 or -1 shift
- final string after all operations

This is difference array.

---

## 16.3 Key Idea

Forward shift:

```
+1
```

Backward shift:

```
-1
```

For range [l, r]:

```
diff[l] += val;
diff[r + 1] -= val;
```

Then each character shifts by prefix sum at its position.

---

## 16.4 C++ Code

```
class Solution {
public:
    string shiftingLetters(string s, vector<vector<int>>& shifts) {
        int n = s.size();

        vector<int> diff(n + 1, 0);

        for (auto& shift : shifts) {
            int l = shift[0];
            int r = shift[1];
            int direction = shift[2];

            int val = direction == 1 ? 1 : -1;
        }
    }
};
```

```

        diff[l] += val;
        diff[r + 1] -= val;
    }

    int running = 0;

    for (int i = 0; i < n; i++) {
        running += diff[i];

        int shiftAmount = ((running % 26) + 26) % 26;

        int oldPosition = s[i] - 'a';
        int newPosition = (oldPosition + shiftAmount) % 26;

        s[i] = 'a' + newPosition;
    }

    return s;
}
};

```

## 16.5 Code Explanation

```
int val = direction == 1 ? 1 : -1;
```

Forward is +1, backward is -1.

```
int shiftAmount = ((running % 26) + 26) % 26;
```

Normalize shifts because running can be negative.

Example:

```
-1 mod 26
```

should mean shift backward by 1, equivalent to +25.

## 16.6 Dry Run

```

s = "abc"
shifts = {
    {0, 1, 0},
    {1, 2, 1},
    {0, 2, 1}
}

```

Operations:

15. [0,1,0]: val = -1

```
diff[0] -= 1
diff[2] += 1
```

16. [1,2,1]: val = +1

```
diff[1] += 1
diff[3] -= 1
```

17. [0,2,1]: val = +1

```
diff[0] += 1
diff[3] -= 1
```

Final running shifts:

|   |   |   |
|---|---|---|
|   |   |   |
| 0 | a | 0 |
| 1 | b | 1 |
| 2 | c | 2 |

Final:

"ace"

## 16.7 Complexity

Time:

$O(n + \text{shifts.size}())$

Space:

$O(n)$

## 16.8 Edge Cases

- many backward shifts
- shifts larger than 26 in total
- full string shift
- single character string
- overlapping shifts

## 16.9 Common Mistakes

Mistake 1: Not normalizing negative modulo

Wrong:

running % 26

Correct:

$((\text{running} \% 26) + 26) \% 26$

Mistake 2: Applying every shift directly

That can be  $O(n * m)$ .

---

## 16.10 Similar Problems

- Range Addition
  - Corporate Flight Bookings
  - Shifting Letters I, different prefix style
- 

# 17. Full Problem Walkthrough 5 — LeetCode 1854: Maximum Population Year

## 17.1 Problem Summary

You are given birth and death years.

Each person is alive from birth year to death year exclusive.

Return the earliest year with maximum population.

Example:

```
logs = {  
    {1993, 1999},  
    {2000, 2010}  
}
```

Output:

```
1993
```

## 17.2 How to Identify This Pattern

Problem involves:

- intervals over years
- count active people
- birth starts population
- death ends population

This is difference array / sweep line.

---

## 17.3 Key Detail: Death Year Exclusive

If log is:

```
{birth, death}
```

Person is alive in years:

```
birth to death - 1
```

So:

```
diff[birth] += 1;  
diff[death] -= 1;
```

## 17.4 C++ Code

```
class Solution {
public:
    int maximumPopulation(vector<vector<int>>& logs) {
        vector<int> diff(2051, 0);

        for (auto& log : logs) {
            int birth = log[0];
            int death = log[1];

            diff[birth]++;
            diff[death]--;
        }

        int current = 0;
        int bestPopulation = 0;
        int bestYear = 1950;

        for (int year = 1950; year <= 2050; year++) {
            current += diff[year];

            if (current > bestPopulation) {
                bestPopulation = current;
                bestYear = year;
            }
        }

        return bestYear;
    }
};
```

## 17.5 Complexity

Years are fixed from 1950 to 2050, so technically  $O(1)$ .  
Generally:

$O(\text{logs.size()} + \text{yearRange})$

Space:

$O(\text{yearRange})$

## 17.6 Edge Cases

- death year is exclusive
- tie should return earliest year

- multiple people born same year
  - many deaths/births same year
- 

## 17.7 Common Mistake

Treating death year as alive.

Wrong:

```
diff[death + 1]--;
```

Correct:

```
diff[death]--;
```

## 17.8 Similar Problems

- Car Pooling
  - Meeting Rooms II
  - My Calendar II
  - Employee Free Time
- 

# 18. Full Problem Walkthrough 6 — LeetCode 1893: Check if All the Integers in a Range Are Covered

## 18.1 Problem Summary

You are given ranges and integers left, right.

Return true if every integer in [left, right] is covered by at least one range.

Example:

```
ranges = {  
  {1, 2},  
  {3, 4},  
  {5, 6}  
}  
left = 2  
right = 5
```

Output:

```
true
```

Numbers 2, 3, 4, 5 are covered.

---

## 18.2 How to Identify This Pattern

Problem asks:

- coverage of integer range
- many inclusive ranges
- small coordinate limits

Use difference array coverage count.

---

## 18.3 C++ Code

```
class Solution {
public:
    bool isCovered(vector<vector<int>>& ranges, int left, int right) {
        vector<int> diff(52, 0);

        for (auto& range : ranges) {
            int l = range[0];
            int r = range[1];

            diff[l] += 1;
            diff[r + 1] -= 1;
        }

        int active = 0;

        for (int i = 1; i <= 50; i++) {
            active += diff[i];

            if (i >= left && i <= right && active <= 0) {
                return false;
            }
        }

        return true;
    }
};
```

## 18.4 Why Size 52?

Constraints usually have values up to 50.

We need  $r + 1$ , so size at least 52.

---

## 18.5 Complexity

Time:  $O(\text{ranges.size()} + 50)$

Space:  $O(1)$ , fixed size.

---

## 18.6 Edge Cases

- range exactly covers boundary
- overlapping ranges
- missing one number
- $\text{left} == \text{right}$

- all ranges outside target
- 

## 18.7 Similar Problems

- Maximum Population Year
  - Car Pooling
  - Range Addition
- 

# 19. Full Problem Walkthrough 7 — LeetCode 995: Minimum Number of K Consecutive Bit Flips

This is a harder difference-array / active-effect problem.

## 19.1 Problem Summary

You are given binary array `nums` and integer `k`.

In one operation, choose a subarray of length `k` and flip every bit.

Return minimum number of flips needed to make all values 1.

If impossible, return -1.

Example:

`nums = {0, 1, 0}`

`k = 1`

Output:

2

## 19.2 How to Identify This Pattern

Each flip affects a range  $[i, i + k - 1]$ .

Need to know whether current index has been flipped odd/even number of times.

This can be tracked using a difference array of active flips.

---

## 19.3 Brute Force Idea

When a zero is found, flip next `k` elements directly.

Time:  $O(n * k)$

Too slow.

---

## 19.4 Optimized Intuition

Maintain `flipParity`, whether current index has been flipped odd number of times.

Use `diff` array to know when a flip effect ends.

At index `i`:

`actualBit = nums[i]` after active flips

If actual bit is 0, we must start a flip at `i`.

Why must?

Because after passing index `i`, no future flip can affect it.

---

## 19.5 C++ Code

```
class Solution {
public:
    int minKBitFlips(vector<int>& nums, int k) {
        int n = nums.size();

        vector<int> diff(n + 1, 0);

        int flipParity = 0;
        int answer = 0;

        for (int i = 0; i < n; i++) {
            flipParity ^= diff[i];

            int currentBit = nums[i];

            if (flipParity == 1) {
                currentBit ^= 1;
            }

            if (currentBit == 0) {
                if (i + k > n) {
                    return -1;
                }

                answer++;

                flipParity ^= 1;
                diff[i + k] ^= 1;
            }
        }

        return answer;
    }
};
```

## 19.6 Code Explanation

```
flipParity ^= diff[i];
```

Apply any flip effect ending/starting at this index.  
Here diff stores parity toggles, not counts.

```
if (flipParity == 1) {  
    currentBit ^= 1;  
}
```

If active flips are odd, actual bit is flipped.

```
if (currentBit == 0)
```

We must flip starting here.

```
diff[i + k] ^= 1;
```

The flip effect ends after index  $i + k - 1$ .

## 19.7 Dry Run

```
nums = {0, 1, 0}  
k = 1
```

| 0 | 0 | 0 | flip [0,0] | 1 |
|---|---|---|------------|---|
| 1 | 0 | 1 | no flip    | 1 |
| 2 | 0 | 0 | flip [2,2] | 2 |

Return 2.

## 19.8 Complexity

Time:  $O(n)$

Space:  $O(n)$

There is also an  $O(1)$  extra-space version using queue/marking, but diff is clearer.

## 19.9 Edge Cases

- impossible near end
- $k = 1$
- $k = n$
- all ones
- all zeroes
- overlapping flips

## 19.10 Common Mistakes

**Mistake 1: Flipping actual array directly**

This becomes  $O(n * k)$ .

**Mistake 2: Not realizing current zero must be flipped immediately**

If current actual bit is 0, only a flip starting at or before current index can fix it. Since earlier positions are already decided, the only choice is starting at current index.

## 19.11 Similar Problems

- Bulb Switcher variants

- Range Addition
  - Greedy range flip problems
- 

## 20. Full Problem Walkthrough 8 — Meeting Rooms II Using Difference/Sweep

This overlaps with heap and sweep line, but it helps show the connection.

### 20.1 Problem Summary

Given meeting intervals, return minimum number of rooms required.

Example:

```
intervals = {  
    {0, 30},  
    {5, 10},  
    {15, 20}  
}
```

Output:

```
2
```

### 20.2 How to Identify This Pattern

Problem asks:

- maximum number of overlapping intervals
- each meeting starts and ends
- active count over time

This is sweep line.

If coordinates are small, difference array works.

If coordinates are large, use sorted events.

---

### 20.3 Event-Based Code

```
int minMeetingRooms(vector<vector<int>>& intervals) {  
    vector<pair<int, int>> events;  
  
    for (auto& interval : intervals) {  
        int start = interval[0];  
        int end = interval[1];  
  
        events.push_back({start, 1});  
        events.push_back({end, -1});  
    }  
  
    sort(events.begin(), events.end());
```

```
int active = 0;
int answer = 0;

for (auto& event : events) {
    active += event.second;
    answer = max(answer, active);
}

return answer;
}
```

## 20.4 Important Tie Issue

If one meeting ends at time 10 and another starts at time 10, they can use the same room.

So end event should be processed before start event at same time.

With pairs {time, delta} and  $-1 < +1$ , sorting naturally processes end first.

---

## 20.5 Complexity

Time:  $O(n \log n)$

Space:  $O(n)$

---

## 20.6 Difference Array Version If Times Are Small

If max time is small:

```
vector<int> diff(maxTime + 1, 0);

for (auto& interval : intervals) {
    diff[start]++;
    diff[end]--;
}
```

Then prefix sum active meetings.

---

# 21. Full Problem Walkthrough 9 — LeetCode 731: My Calendar II Preview

This is more advanced but useful.

## 21.1 Problem Summary

Implement calendar booking.

A booking can overlap with one existing booking, but triple booking is not allowed.

Return false if new event causes triple booking.

---

## 21.2 Pattern

Use sweep-line events.

For each booking [start, end):

```
events[start] += 1;  
events[end] -= 1;
```

Then scan active count.

If active count reaches 3, reject booking and undo changes.

---

## 21.3 C++ Code

```
class MyCalendarTwo {  
private:  
    map<int, int> events;  
  
public:  
    MyCalendarTwo() {  
    }  
  
    bool book(int start, int end) {  
        events[start]++;  
        events[end]--;  
  
        int active = 0;  
  
        for (auto& p : events) {  
            active += p.second;  
  
            if (active >= 3) {  
                events[start]--;  
                events[end]++;  
  
                if (events[start] == 0) {  
                    events.erase(start);  
                }  
  
                if (events[end] == 0) {  
                    events.erase(end);  
                }  
  
                return false;  
            }  
        }  
    }  
}
```

```
        return true;
    }
};
```

## 21.4 Why map Instead of Difference Array?

Coordinates can be large.

We only store event points.

This is sparse difference array / sweep line.

---

## 21.5 Complexity

Each booking scans events.

If there are  $n$  bookings:

Time per booking:  $O(n)$

Space:  $O(n)$

---

## 21.6 Similar Problems

- My Calendar I
  - My Calendar III
  - Meeting Rooms II
  - Car Pooling
- 

# 22. Full Problem Walkthrough 10 — LeetCode 1674: Minimum Moves to Make Array Complementary

This is a harder difference-array problem.

## 22.1 Problem Summary

You are given even-length array `nums` and integer `limit`.

For every pair:

```
nums[i], nums[n - 1 - i]
```

You want all pair sums to become the same target sum.

In one move, you can replace one number with any value from 1 to `limit`.

Return minimum moves.

---

## 22.2 Why Difference Array Applies

For each pair  $(a, b)$ , possible target sums range from 2 to  $2 * \text{limit}$ .

For each target sum, that pair needs:

- 0 moves if  $\text{target} = a + b$
- 1 move if target can be achieved by changing one number
- 2 moves otherwise

We need total moves over all target sums.

Difference array can add move costs over ranges.

---

## 22.3 Pair Contribution

Let:

```
a = min(nums[i], nums[j])
b = max(nums[i], nums[j])
```

For this pair:

- 2 moves baseline for all sums  $[2, 2 \cdot \text{limit}]$
- 1 move possible for sums  $[a + 1, b + \text{limit}]$
- 0 moves for sum  $a + b$

We can update difference array over cost ranges.

---

## 22.4 C++ Code

```
class Solution {
public:
    int minMoves(vector<int>& nums, int limit) {
        int n = nums.size();

        vector<int> diff(2 * limit + 2, 0);

        for (int i = 0; i < n / 2; i++) {
            int x = nums[i];
            int y = nums[n - 1 - i];

            int a = min(x, y);
            int b = max(x, y);

            int sum = a + b;

            // baseline: 2 moves for all sums
            diff[2] += 2;
            diff[2 * limit + 1] -= 2;

            // 1 move range
            diff[a + 1] -= 1;
            diff[b + limit + 1] += 1;

            // 0 move exact sum
            diff[sum] -= 1;
            diff[sum + 1] += 1;
        }
    }
};
```

```

int answer = INT_MAX;
int current = 0;

for (int target = 2; target <= 2 * limit; target++) {
    current += diff[target];
    answer = min(answer, current);
}

return answer;
}
};

```

## 22.5 Explanation

For every pair, we add a piecewise cost function over possible target sums. Difference array lets us add these piecewise ranges efficiently. Instead of checking every target for every pair, we mark ranges and prefix once.

## 22.6 Complexity

Let  $n = \text{nums.size}()$  and possible sums go up to  $2 * \text{limit}$ .  
Time:

$O(n + \text{limit})$

Space:

$O(\text{limit})$

## 22.7 Why This Is Hard

This problem requires seeing that every pair contributes a range-based cost function. Difference array is used not to update original array, but to accumulate costs over possible target sums.

## 22.8 Similar Problems

- Minimum Moves to Make Array Complementary
- Sweep-line cost accumulation problems
- Range-add optimization problems

# 23. Mini Problem 1 — Difference Array With Existing Array

Suppose:

```

nums = {5, 5, 5, 5}
updates = {

```

```
{1, 2, 3}
}
```

Add 3 to indices 1 through 2.  
You can build diff from original array:

```
diff[0] = nums[0];

for (int i = 1; i < n; i++) {
    diff[i] = nums[i] - nums[i - 1];
}
```

Then apply update:

```
diff[1] += 3;
diff[3] -= 3;
```

Reconstruct with prefix.

## 24. Mini Problem 2 — Inclusive vs Exclusive Ranges

This is a common source of bugs.

### Inclusive Range [l, r]

Effect includes index r.

Use:

```
diff[l] += val;
diff[r + 1] -= val;
```

Examples:

- Corporate Flight Bookings
- Range Addition
- Shifting Letters II

### Exclusive End Range [start, end)

Effect stops before end.

Use:

```
diff[start] += val;
diff[end] -= val;
```

Examples:

- Car Pooling
- Meeting intervals
- Population year with death exclusive

## 25. Mini Problem 3 — Maximum Overlap With Small Coordinates

Given intervals and small coordinate range, find maximum overlap.

```
int maxOverlap(vector<vector<int>>& intervals, int maxCoord) {
    vector<int> diff(maxCoord + 2, 0);

    for (auto& interval : intervals) {
        int l = interval[0];
        int r = interval[1];

        diff[l]++;
        diff[r + 1]--;
    }

    int active = 0;
    int best = 0;

    for (int i = 0; i <= maxCoord; i++) {
        active += diff[i];
        best = max(best, active);
    }

    return best;
}
```

This assumes intervals are inclusive.

For exclusive end intervals, use:

```
diff[end]--;
```

## 26. Mini Problem 4 — 2D Difference Array Concept

Suppose you need to add val to a rectangle:

```
top-left: (r1, c1)
bottom-right: (r2, c2)
```

inclusive.

Mark:

```
diff[r1][c1] += val;
diff[r2 + 1][c1] -= val;
```

```
diff[r1][c2 + 1] -= val;  
diff[r2 + 1][c2 + 1] += val;
```

Then reconstruct using 2D prefix sum.  
This is useful for advanced grid update problems.

## 27. Difference Array vs Sweep Line

Difference array is basically sweep line on small integer coordinates.

|                                 |                             |
|---------------------------------|-----------------------------|
|                                 |                             |
| Coordinate range small          | Difference array            |
| Coordinate range huge/sparse    | Sweep line with map         |
| Need sorted event processing    | Sweep line                  |
| Need final value at every index | Difference array            |
| Need maximum active intervals   | Either                      |
| Need dynamic online events      | Ordered map / balanced tree |

## 28. Difference Array vs Segment Tree

|                      |                     |  |
|----------------------|---------------------|--|
|                      |                     |  |
| Range updates        | Yes                 |  |
| Final reconstruction | Yes                 |  |
| Online range queries | No                  |  |
| Mixed update/query   | Not ideal           |  |
| Complexity           | Simple $O(n + m)$   |  |
| Implementation       | Easy                |  |
| Lazy propagation     | Not needed          |  |
| Dynamic queries      | Segment tree better |  |

Use Difference Array when all updates are known first.  
Use Segment Tree when updates and queries are mixed.

## 29. Common Edge Cases

## 29.1 Update Starts at 0

```
diff[0] += val;
```

Works.

## 29.2 Update Ends at Last Index

If using  $n + 1$  diff:

```
diff[n] -= val;
```

Safe.

If using size  $n$ , must check boundary.

## 29.3 Negative Updates

Difference array handles negative values naturally.

## 29.4 Overlapping Updates

Overlaps are automatically accumulated by prefix sum.

## 29.5 No Updates

Final array remains all zero, or original array if initialized.

## 29.6 Large Values

Use long long if cumulative value can exceed int.

## 29.7 Inclusive vs Exclusive End

Always read problem statement carefully.

This is the most common issue.

## 29.8 Coordinate Limits

If maximum coordinate is huge, do not allocate massive array.

Use map/sweep line.

# 30. Common Mistakes and Wrong Code

## Mistake 1 — Forgetting $r + 1$

Wrong:

```
diff[l] += val;  
diff[r] -= val;
```

For inclusive  $[l, r]$ , this stops too early.

Correct:

```
diff[l] += val;  
diff[r + 1] -= val;
```

## Mistake 2 — Using $r + 1$ for Exclusive End

For  $[start, end)$ , wrong:

```
diff[end + 1] -= val;
```

Correct:

```
diff[end] -= val;
```

## Mistake 3 — Out-of-Bounds at $r + 1$

If diff size is  $n$ , then  $r + 1$  may go out of bounds.

Use:

```
vector<int> diff(n + 1, 0);
```

## Mistake 4 — Forgetting Prefix Reconstruction

After marking diff, you must prefix sum it.

Marks alone are not final values.

## Mistake 5 — Confusing Difference Array With Prefix Sum

Difference array stores changes.

Prefix sum reconstructs values.

## Mistake 6 — Not Handling 1-Indexed Input

Corporate Flight Bookings uses 1-indexed flight numbers.

Convert to 0-index.

## Mistake 7 — Using Difference Array With Huge Coordinates

If coordinate is up to  $10^9$ , array allocation is impossible.

Use ordered map / coordinate compression.

# 31. Interview Explanation Templates

## 31.1 Basic Range Update

Since each operation adds a value over a continuous range, updating every element would be too slow. I use a difference array where I mark the start of the effect with  $+val$  and the position after the range with  $-val$ . A final prefix sum reconstructs the actual values.

## 31.2 Car Pooling / Active Count

Each trip adds passengers at the pickup location and removes them at the drop-off location. By processing these changes as a difference array, the prefix sum gives the number of active passengers at each location. If it ever exceeds capacity, return false.

---

### 31.3 String Shifting

Each shift operation affects a range of characters. I convert forward shifts to +1 and backward shifts to -1, mark them in a difference array, then prefix sum to get the net shift at each character.

---

### 31.4 Sweep-Line Relation

A difference array is like a sweep line over integer coordinates. We record start and end events, then scan from left to right to maintain the active value.

---

## 32. Complexity Patterns

| Basic range updates           | $O(n + m)$                           | $O(n)$                |
|-------------------------------|--------------------------------------|-----------------------|
| Corporate Flight Bookings     | $O(n + \text{bookings})$             | $O(n)$                |
| Car Pooling small coordinates | $O(\text{trips} + \text{maxCoord})$  | $O(\text{maxCoord})$  |
| Shifting Letters II           | $O(n + \text{shifts})$               | $O(n)$                |
| Maximum Population Year       | $O(\log s + \text{yearRange})$       | $O(\text{yearRange})$ |
| Coverage check small range    | $O(\text{ranges} + \text{maxCoord})$ | $O(\text{maxCoord})$  |
| Sweep line with map           | $O(m \log m)$                        | $O(m)$                |
| 2D difference                 | $O(R * C + \text{updates})$          | $O(R * C)$            |
| Complementary array moves     | $O(n + \text{limit})$                | $O(\text{limit})$     |

## 33. Time-Boxed Practice Guidance

### Easy Difference Array Problems

Target:

10–15 minutes

Examples:

- Maximum Population Year
  - Check if All Integers in Range Are Covered
  - Range Addition
- 

### Medium Difference Array Problems

Target:

20–35 minutes

Examples:

- Corporate Flight Bookings
- Car Pooling
- Shifting Letters II

## Harder Difference/Sweep Problems

Target:

35–60 minutes

Examples:

- Minimum Number of K Consecutive Bit Flips
- My Calendar II
- Minimum Moves to Make Array Complementary
- Stamping the Grid

# 34. Expanded Practice Set

## 34.1 Must-Do Easy

- 1.1. Maximum Population Year
- 2.2. Check if All the Integers in a Range Are Covered
- 3.3. Range Addition, premium-style classic
- 4.4. Points That Intersect With Cars, can be solved by difference/counting
- 5.5. Corporate Flight Bookings

## 34.2 Must-Do Medium

- 6.6. Corporate Flight Bookings
- 7.7. Car Pooling
- 8.8. Shifting Letters II
- 9.9. Minimum Number of K Consecutive Bit Flips
- 10.10. Meeting Rooms II, sweep-line version
- 11.11. My Calendar II
- 12.12. My Calendar III
- 13.13. Divide Intervals Into Minimum Number of Groups, sweep/heap overlap

## 34.3 Strong Interview Practice

- 14.14. Minimum Number of K Consecutive Bit Flips
- 15.15. Minimum Moves to Make Array Complementary
- 16.16. Number of Flowers in Full Bloom, sweep/binary search
- 17.17. Describe the Painting, sweep line
- 18.18. The Skyline Problem, advanced sweep line
- 19.19. Stamping the Grid, 2D prefix/difference style

## 34.4 Hard / Stretch

- 20.20. Minimum Moves to Make Array Complementary
- 21.21. Stamping the Grid

- 22.22. The Skyline Problem
  - 23.23. My Calendar III
  - 24.24. Rectangle Area II, advanced sweep line
  - 25.25. Increment Submatrices by One, 2D difference
- 

## 35. Suggested Study Order

### Step 1 — Basic Concept

- 18. Range Addition
- 19. Corporate Flight Bookings

Goal:

Understand:

```
diff[l] += val;  
diff[r + 1] -= val;
```

### Step 2 — Active Count / Timeline

- 20. Maximum Population Year
- 21. Car Pooling
- 22. Check if All Integers Covered

Goal:

Understand start/end events and active count.

---

### Step 3 — String Range Updates

- 23. Shifting Letters II

Goal:

Learn difference array with modulo and negative values.

---

### Step 4 — Greedy + Difference

- 24. Minimum Number of K Consecutive Bit Flips

Goal:

Learn active range effects while scanning.

---

### Step 5 — Sweep Line Bridge

- 25. Meeting Rooms II, sweep version
- 26. My Calendar II
- 27. My Calendar III

Goal:

Understand difference array as sweep-line events.

---

### Step 6 — Advanced Range Cost

- 28. Minimum Moves to Make Array Complementary

Goal:

Learn difference array over possible answer space.

---

## 36. Pattern Decision Table

|                        |                  |                              |
|------------------------|------------------|------------------------------|
|                        |                  |                              |
| Add val to [l,r]       | difference array | diff[l]+=val, diff[r+1]-=val |
| Flights 1 to n         | 1-index convert  | l=first-1, r=last-1          |
| Trip [from,to)         | exclusive end    | diff[from]+=p, diff[to]-=p   |
| Population birth/death | death exclusive  | diff[birth]++, diff[death]-- |
| String range shift     | diff + modulo    | prefix shift per char        |
| Count active intervals | diff or sweep    | prefix active count          |
| Huge coordinates       | sweep map        | map[time]+=delta             |
| Rectangle updates      | 2D diff          | four corner updates          |
| Range flip effects     | parity diff      | XOR active state             |

## 37. Final Cheat Sheet

### Basic Difference Array

```
vector<int> diff(n + 1, 0);

for (auto& update : updates) {
    int l = update[0];
    int r = update[1];
    int val = update[2];

    diff[l] += val;
    diff[r + 1] -= val;
}

vector<int> result(n);
int running = 0;

for (int i = 0; i < n; i++) {
    running += diff[i];
    result[i] = running;
}
```

### Inclusive Range [l, r]

```
diff[l] += val;  
diff[r + 1] -= val;
```

## Exclusive End Range [start, end)

```
diff[start] += val;  
diff[end] -= val;
```

## Coverage Count

```
diff[l]++;  
diff[r + 1]--;  
  
active += diff[i];
```

## Car Pooling

```
diff[from] += passengers;  
diff[to] -= passengers;
```

## Corporate Flight Bookings

```
int l = first - 1;  
int r = last - 1;  
  
diff[l] += seats;  
diff[r + 1] -= seats;
```

## Shifting Letters II

```
int val = direction == 1 ? 1 : -1;  
  
diff[l] += val;  
diff[r + 1] -= val;
```

Then:

```
running += diff[i];  
shift = ((running % 26) + 26) % 26;
```

## Sweep Line Map

```
map<int, int> events;  
  
events[start]++;
```

```

events[end]--;

int active = 0;

for (auto& p : events) {
    active += p.second;
}

```

## 2D Difference Update

```

diff[r1][c1] += val;
diff[r2 + 1][c1] -= val;
diff[r1][c2 + 1] -= val;
diff[r2 + 1][c2 + 1] += val;

```

# 38. What to Master Before Chapter 7

Before moving to Sorting-Based Patterns, make sure you can:

- Explain what a difference array stores
- Derive original array from diff using prefix sum
- Apply inclusive range update in  $O(1)$
- Handle  $r + 1$  safely
- Distinguish inclusive  $[l, r]$  from exclusive  $[start, end)$
- Solve Corporate Flight Bookings
- Solve Car Pooling
- Solve Shifting Letters II
- Use diff for coverage/counting problems
- Know when coordinates are too large for array diff
- Understand difference array as sweep-line events
- Explain  $O(n + m)$  complexity
- Know when segment tree is needed instead

# 39. Core Takeaway

Difference Array is the opposite partner of Prefix Sum.

Prefix Sum helps answer many range queries quickly.

Difference Array helps apply many range updates quickly.

The core operation is:

```

diff[l] += val;
diff[r + 1] -= val;

```

for inclusive ranges.

Then:

```
running += diff[i]
```

reconstructs the final values.

The biggest idea is:

A range update only changes behavior at two boundary points: where the effect starts and where it stops.

If you can mark those two points, you can avoid updating every element inside the range.

That is the power of Difference Array.